Proyecto Final de Ciclo

WEARO

Tu armario inteligente en el bolsillo

Autor: Claudio Pastor Corbalán
Curso académico: 2025-2026
Centro: Florida Campus Alzira
Ciclo Formativo: Desarrollo de Aplicaciones Multiplataforma

Marzo 2026

Índice

Índice de figuras 2
1. Resumen del proyecto 3
Abstract 3
2. Justificacion y objetivos del proyecto 4
2.1. Situacion de partida 4
2.2. A quien se dirige 4
2.3. Objetivos del proyecto 4
3. Desarrollo del proyecto 5
3.1. Analisis del mercado y modelo de negocio 5
3.2. Metodologias utilizadas 6
3.3. Descripcion de los componentes de la aplicacion 7
3.4. Resultados obtenidos 9
4. Conclusiones 10
5. Lineas futuras de trabajo 11
6. Anexo: Diario de trabajo 12
Semana 1: 23 - 27 de febrero de 2026 12
Semana 2: 3 - 7 de marzo de 2026 13
7. Bibliografia 15

Índice de figuras

Figura 1. Arquitectura general del sistema 7
Figura 2. Stack tecnológico 7
Figura 3. Estructura del proyecto 8
Figura 4. Endpoints principales de la API 9

1. Resumen del proyecto

Wearo es una aplicación móvil completa de gestión de armario personal y red social de moda. El proyecto consiste en el desarrollo de una plataforma que permite a los usuarios catalogar sus prendas de ropa, crear combinaciones de outfits, planificar que ponerse cada día según el clima, y compartir sus looks con una comunidad de usuarios.

El sistema está formado por tres componentes principales: una aplicación móvil desarrollada con React Native y Expo, un backend en forma de API REST construido con Node.js y Express, y una base de datos PostgreSQL para el almacenamiento persistente de los datos. Además, integra servicios de inteligencia artificial (Google Gemini) para recomendaciones personalizadas de outfits basadas en el clima, un sistema de mensajería directa entre usuarios, y un panel de administración completo para la moderación de contenido.

El proyecto ha sido desplegado en producción utilizando Railway como plataforma de hosting en la nube, permitiendo el acceso al backend y la base de datos desde cualquier dispositivo conectado a Internet.

Abstract

Wearo is a comprehensive mobile application for personal wardrobe management and fashion social networking. The project involves the development of a platform that allows users to catalogue their clothing items, create outfit combinations, plan what to wear each day based on weather conditions, and share their looks with a community of users.

The system is composed of three main components: a mobile application developed with React Native and Expo, a backend REST API built with Node.js and Express, and a PostgreSQL database for persistent data storage. Additionally, it integrates artificial intelligence services (Google Gemini) for personalized outfit recommendations based on weather, a direct messaging system between users, and a complete administration panel for content moderation.

The project has been deployed in production using Railway as a cloud hosting platform, enabling access to the backend and database from any device connected to the Internet.

2. Justificación y objetivos del proyecto

2.1. Situación de partida

En la actualidad, la gestión del armario personal es uno de los problemas cotidianos más comunes. Según estudios recientes, una persona media utiliza solo el 20% de su armario de manera habitual, y dedica una media de 15-20 minutos diarios a decidir que ponerse (Jaeger, 2019). Además, la falta de organización lleva a compras duplicadas y a un desaprovechamiento de las prendas que ya se poseen.

Las soluciones existentes en el mercado se limitan generalmente a una funcionalidad básica de catálogo, sin integrar aspectos sociales, inteligencia artificial o planificación basada en condiciones climáticas. Esto deja un espacio de oportunidad para una aplicación que combine todas estas funcionalidades en una sola plataforma.

2.2. A quien se dirige

La aplicación está dirigida a cualquier persona interesada en la moda y la organización personal, con un público objetivo principal de jóvenes entre 18 y 35 años que utilizan smartphones de manera habitual. También se dirige a creadores de contenido de moda que buscan una plataforma para compartir sus looks.

2.3. Objetivos del proyecto

Los objetivos principales del proyecto son:

- Desarrollar una aplicación móvil multiplataforma (iOS y Android) funcional y completa.
- Implementar un sistema de catalogación de prendas de ropa con detección automática de color.
- Crear un motor de recomendación de outfits basado en inteligencia artificial y condiciones climáticas.
- Diseñar una red social integrada con feed, likes, comentarios y mensajería directa.
- Implementar un calendario de planificación de looks diario.
- Proporcionar estadísticas de uso del armario para optimizar las decisiones de compra.
- Desarrollar un panel de administración para la moderación de contenido.

- Desplegar el sistema en un entorno de producción en la nube.

3. Desarrollo del proyecto

3.1. Análisis del mercado y modelo de negocio

Competidores principales

Se ha realizado un análisis de las aplicaciones similares disponibles en el mercado:

- **Cladwell:** Aplicación de gestión de armario con recomendaciones básicas. Limitada a la catalogación y sugerencias simples. Modelo de suscripción mensual.
- **Acloset:** Armario digital con creación de outfits. Carece de funcionalidades sociales e inteligencia artificial. Freemium con compras in-app.
- **Combyne:** Red social de moda con catálogo. Enfocada en la parte social, pero con un sistema de catalogación limitado. Gratuita con anuncios.
- **Smart Closet:** Gestión de armario con sugerencias. Sin integración de clima ni mensajería. Freemium.

Valor añadido de Wearo

El valor diferencial de Wearo respecto a los competidores reside en la combinación integral de funcionalidades que ninguna otra aplicación ofrece de manera conjunta:

- Inteligencia artificial con contexto climático: las recomendaciones no son genéricas, sino basadas en el clima real de la ciudad del usuario.
- Red social completa integrada: feed, likes, comentarios con respuestas y mensajería directa, todo dentro de la misma aplicación.
- Detección automática de color: utiliza ColorThief para identificar el color dominante de cada prenda fotografiada.
- Transcripción de audio: mensajes de voz convertidos a texto gracias a Groq Whisper.
- Panel de administración completo para la moderación del contenido de la comunidad.

Modelo de negocio potencial

Wearo podría explotarse comercialmente mediante un modelo freemium:

- Versión gratuita: catalogación básica, creación de outfits y acceso al feed social.
- Versión premium: recomendaciones IA ilimitadas, estadísticas avanzadas y almacenamiento ilimitado de prendas.
- Colaboraciones con marcas: integración de catálogos de marcas para sugerencias de compra personalizadas.
- Publicidad no intrusiva en el feed social para marcas de moda.

3.2. Metodologías utilizadas

Para el desarrollo del proyecto se ha seguido una metodología ágil basada en Scrum adaptado a un equipo unipersonal. El trabajo se ha organizado en sprints semanales con objetivos concretos, permitiendo una iteración rápida y la adición progresiva de funcionalidades.

Control de versiones

Se ha utilizado Git como sistema de control de versiones, con un repositorio alojado en GitHub. La estrategia de ramas sigue el modelo Git Flow simplificado con dos ramas principales:

- main: rama de producción, contiene el código estable desplegado en Railway.
- develop: rama de integración donde se fusionan las nuevas funcionalidades antes de pasar a producción.

Fases del desarrollo

El proyecto se ha desarrollado en las siguientes fases:

- Fase 1: Diseño de la arquitectura y configuración del entorno de desarrollo.
- Fase 2: Desarrollo del backend (API REST, base de datos, autenticación).
- Fase 3: Desarrollo del frontend (pantallas principales, navegación, estado).
- Fase 4: Integración de servicios externos (IA, clima, detección de color).
- Fase 5: Implementación de funcionalidades sociales (feed, mensajería, perfiles).
- Fase 6: Panel de administración y moderación.
- Fase 7: Despliegue en la nube y pruebas finales.

Herramientas de desarrollo

- Visual Studio Code como editor de código principal.
- Expo Go para las pruebas en dispositivo móvil durante el desarrollo.
- Postman para las pruebas de los endpoints de la API.
- pgAdmin para la gestión de la base de datos PostgreSQL.

3.3. Descripción de los componentes de la aplicación

Arquitectura general

La aplicación sigue una arquitectura cliente-servidor de tres capas (Figura 1):

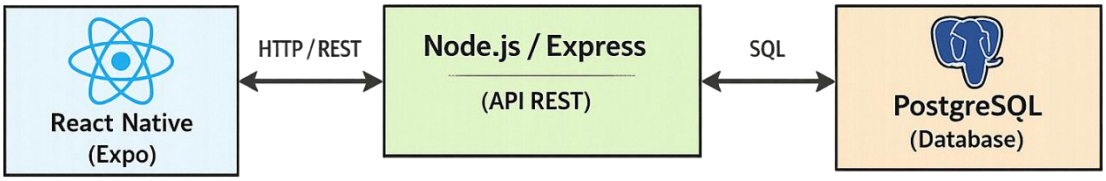

Figura 1. Arquitectura general del sistema

El frontend (capa de presentación) es una aplicación móvil React Native que se comunica con el backend mediante peticiones HTTP a una API REST. El backend (capa de lógica de negocio) procesa las peticiones, interactúa con la base de datos y con servicios externos como Google Gemini u OpenWeatherMap. La base de datos PostgreSQL (capa de persistencia) almacena todos los datos de manera relacional.

Stack tecnológico

La siguiente tabla resume las tecnologías utilizadas en cada capa del sistema (Figura 2):

Capa	Tecnología
Aplicación móvil	React Native (Expo)
Gestión de estado	Redux Toolkit
Backend	Node.js + Express
Base de datos	PostgreSQL
Autenticación	JWT + bcrypt
Inteligencia Artificial	Google Gemini, Groq Whisper
Clima	OpenWeatherMap API
Procesamiento de imágenes	Multer + ColorThief
Despliegue	Railway (backend + PostgreSQL)
Despliegue	Railway (backend + PostgreSQL)

Figura 2. Stack tecnológico del proyecto

Frontend: Aplicación móvil

La aplicación móvil esta desarrollada con React Native y Expo, permitiendo su ejecución tanto en iOS como en Android desde un único código fuente. Consta de 19 pantallas:

- HomeScreen: pantalla principal con clima, sugerencias IA y estadísticas rápidas.
- WardrobeScreen: gestión del armario con filtros por categoría, temporada y color.
- AddGarmentScreen: formulario de alta de prendas con captura fotográfica.
- CreateOutfitScreen: creación de outfits seleccionando prendas del armario.
- SocialScreen: feed social con modos Descubrir y Siguiendo.
- ChatScreen: mensajería directa con texto, fotos y audio.
- CalendarScreen: calendario mensual para planificar outfits.
- StatsScreen: estadísticas con gráficos del armario.
- ProfileScreen: perfil de usuario con seguidores y publicaciones.
- AdminScreen: panel de administración y moderación.

Estructura del proyecto

La organización del código sigue una estructura modular y clara (Figura 3):

Figura 3. Estructura del proyecto

Backend: API REST

El backend expone una API REST con los siguientes grupos de endpoints principales (Figura 4):

Ruta	Descripción
POST /api/v1/auth/register	Registro de usuario
POST /api/v1/auth/login	Inicio de sesión
GET /api/v1/garments	Listar prendas
POST /api/v1/garments	Crear prenda
GET /api/v1/outfits	Listar outfits
GET /api/v1/social/feed	Feed social
POST /api/v1/ai/recommend	Recomendación IA
POST /api/v1/ai/chat	Chat con IA
GET /api/v1/messages/ conversations	Conversaciones
GET /api/v1/calendar:/:year /:	Estadisticas
GET /api/v1/stats	Panel admin

Figura 4. Endpoints principales de la API

Base de datos

La base de datos PostgreSQL contiene 14 tablas relacionales: users, refresh_tokens, garments, outfits, outfit_garments, calendar_entries, shared_outfits, likes, comments, follows, follow_requests, conversations, messages y support_tickets. El esquema completo se encuentra en el fichero setup.sql del proyecto.

Integraciones externas

El sistema integra diversos servicios externos:

- **Google Gemini:** motor de IA para recomendaciones de outfits, chat conversacional y sugerencias de compra.
- **OpenWeatherMap:** datos meteorológicos en tiempo real para las recomendaciones basadas en clima.
- **Groq Whisper:** transcripción de audio a texto para los mensajes de voz.
- **Nominatim:** autocompletado de ciudades para la selección de ubicación.
- **ColorThief:** detección automática del color dominante en fotografías de prendas.

3.4. Resultados obtenidos

El desarrollo del proyecto ha resultado en una aplicación completamente funcional con las siguientes cifras:

- 19 pantallas en el frontend cubren todas las funcionalidades planificadas.
- 14 tablas en la base de datos con un esquema relacional completo.
- 10 grupos de rutas en el backend con más de 50 endpoints.
- Integración con 5 servicios externos (Gemini, OpenWeatherMap, Groq, Nominatim, ColorThief).
- Sistema de mensajería en tiempo real con soporte para texto, imágenes y audio.
- Despliegue en producción en Railway con acceso público vía HTTPS.

La aplicación ha sido probada en dispositivos iOS y Android mediante Expo Go, demostrando un rendimiento fluido y una experiencia de usuario consistente en ambas plataformas. El backend desplegado en Railway responde correctamente a todas las peticiones con tiempos de respuesta inferiores a 500ms para la mayoría de las operaciones.

Las funcionalidades de inteligencia artificial han demostrado ser capaces de generar recomendaciones pertinentes basadas en el contenido real del armario del usuario y las condiciones climáticas de su ubicación, añadiendo un valor diferencial respecto a las soluciones existentes en el mercado.

4. Conclusiones

El proyecto Wearo ha conseguido desarrollar con éxito una aplicación móvil completa que cumple todos los objetivos planteados inicialmente. Se ha creado una plataforma que no solo funciona como armario digital, sino que integra de manera cohesionada funcionalidades sociales, inteligencia artificial y planificación basada en el clima.

Los principales logros del proyecto son:

- Se ha desarrollado una aplicación multiplataforma funcional con React Native y Expo que opera tanto en iOS como en Android.
- Se ha implementado con éxito la integración de inteligencia artificial mediante Google Gemini, proporcionando recomendaciones personalizadas y contextualizadas.

- Se ha creado una red social completa dentro de la aplicación, con feed, interacciones y mensajería directa.
- Se ha desplegado el sistema en un entorno de producción en la nube (Railway), demostrando la viabilidad del proyecto como servicio accesible en Internet.
- Se ha diseñado una base de datos relacional robusta que soporta todas las funcionalidades requeridas de manera eficiente.

Durante el desarrollo se han encontrado diversos retos técnicos, como la gestión de los ficheros multimedia en un entorno de nube con sistema de ficheros efímero, la implementación de un sistema de autenticación seguro con tokens de refresco, y la integración de múltiples servicios externos de manera fiable. Todos estos retos han sido resueltos satisfactoriamente, contribuyendo al aprendizaje técnico del proyecto.

En conclusión, Wearo demuestra que es posible crear una solución completa y funcional que aporte valor real a los usuarios en la gestión de su armario personal, combinando tecnologías modernas con un diseño centrado en la experiencia de usuario.

5. Líneas futuras de trabajo

El proyecto Wearo tiene un potencial de expansión considerable. Las siguientes líneas de trabajo se consideran prioritarias para futuras iteraciones:

Mejoras técnicas

- Implementar notificaciones push en tiempo real para mensajes, likes y nuevos seguidores.
- Migrar el almacenamiento de ficheros a un servicio como Amazon S3 o Cloudflare R2 para mayor escalabilidad.
- Anadir WebSockets para la mensajería en tiempo real, sustituyendo el sistema actual de polling.
- Implementar caching con Redis para mejorar los tiempos de respuesta del backend.

Nuevas funcionalidades

- Reconocimiento automático de prendas de ropa a partir de fotografías mediante visión artificial.

- Sistema de sugerencias de compra con enlaces directos a tiendas online.
- Creación de outfits colaborativos entre amigos.
- Herramientas para creadores de contenido: analytics de publicaciones, programación de posts.
- Modo de viaje: planificación de armario para viajes con recomendación de prendas según el destino.

Escalabilidad y monetización

- Implementar el modelo freemium con suscripción premium para funcionalidades avanzadas.
- Establecer colaboraciones con marcas de moda para un marketplace integrado.
- Publicación en las tiendas oficiales (App Store y Google Play).
- Internacionalización de la aplicación a múltiples idiomas.

6. Anexo: Diario de trabajo

A continuación, se presenta el diario de trabajo detallado del desarrollo del proyecto durante las dos últimas semanas de trabajo.

Semana 1: 23 - 27 de febrero de 2026

Lunes 23	Martes 24	Miércoles 25	Jueves 26	Viernes 27
Revisión arquitectura backend. Bugs autenticación.	Fix interceptor 401. Pruebas login.	Panel admin: separar posts eliminados Wearo/usuarios.	Identidad compartida admins. Follow/unfollow Wearo.	Pruebas completas. Estilos AdminScreen. Revisión visual.

Lunes 23 de febrero

Inicio de la fase de pulido del proyecto. Revisión completa de la arquitectura del backend: controladores, modelos y rutas. Identificación de bugs pendientes en el sistema de autenticación (el interceptor 401 de la API interceptaba las respuestas de login, mostrando "Error de conexión" en lugar de "Credenciales incorrectas"). Documentación de los problemas detectados y planificación de las correcciones.

Martes 24 de febrero

Corrección del bug de autenticación: modificación del interceptor de refresco de tokens en api.js para excluir las rutas /auth/ del proceso de refresh automático. Con esta corrección, los errores 401 del login llegan directamente al servicio de autenticación con el mensaje correcto del servidor. Pruebas exhaustivas del flujo de login con credenciales correctas e incorrectas.

Miércoles 25 de febrero

Trabajo en el panel de administración. Implementación de la separación de publicaciones eliminadas: ahora el sistema distingue entre publicaciones de Wearo (admins) y publicaciones de usuarios normales. Modificación de getMyActions y getAdminUserStats en el modelo de administración para devolver ambos tipos por separado. Actualización del frontend con subpestañas dedicadas.

Jueves 26 de febrero

Implementación de la identidad compartida de administradores para el perfil de Wearo. Cuando un usuario sigue la cuenta oficial "Wearo", ahora el follow se aplica a todos los administradores simultáneamente. Modificación de followUser, unfollowUser, getFollowers y getFollowing en el controlador de perfiles. Creación de nuevas funciones getFollowersOfAllAdmins y getFollowingOfAllAdmins en el modelo de follows.

Viernes 27 de febrero

Pruebas completas de todas las correcciones implementadas durante la semana. Adición de los estilos CSS que faltaban en AdminScreen (subTabs, subTab, subTabText). Revisión de la coherencia visual de la aplicación en modo claro y modo oscuro. Documentación del código con comentarios aclaratorios en las funciones más complejas.

Semana 2: 3 - 7 de marzo de 2026

Lunes 3	Martes 4	Miércoles 5	Jueves 6	Viernes 7
Investigación hosting. Config Railway. DB y servidor.	Entorno producción. .env, .gitignore, apiUrl configurable.	Deploy Railway. Sync ramas. Merge develop a main.	PostgreSQL producción. setup.sql. Migración datos.	Imágenes repo. EAS Build. IPA. README. Memoria.

Lunes 3 de marzo

Inicio de la fase de despliegue en la nube. Investigación de plataformas de hosting: comparación entre Railway, AWS, Heroku y Render. Selección de Railway por su simplicidad e integración nativa con PostgreSQL. Modificación de db.js para soportar la variable DATABASE_URL (cadena de conexión completa con SSL). Cambio del binding del servidor a 0.0.0.0 para aceptar conexiones externas en contenedores.

Martes 4 de marzo

Configuración del entorno de producción. Creación del fichero .env.example como referencia de variables de entorno. Configuración del .gitignore para proteger secretos y ficheros sensibles. Creación de .gitkeep en los directorios de uploads para mantener la estructura en el repositorio. Modificación de api.js en el frontend para soportar una URL configurable mediante Expo Constants (extra.apiUrl en app.json).

Miércoles 5 de marzo

Despliegue en Railway. Creación del proyecto en Railway, conexión con el repositorio de GitHub, configuración del Root Directory como backend. Resolución de problemas con la rama de despliegue (Railway desplegaba desde main con código antiguo, era necesario sincronizar con develop). Merge de develop a main y push forzado para actualizar el código en producción.

Jueves 6 de marzo

Configuración de la base de datos en producción. Creación del servicio PostgreSQL en Railway. Creación del fichero setup.sql completo con las 14 tablas y todas las columnas necesarias. Ejecución del setup.sql contra la base de datos de Railway. Migración de los datos locales (6 usuarios, 11 prendas, 6 outfits, 4 publicaciones, 28 mensajes) a la base de datos en producción mediante un script personalizado de Node.js. Verificación de que todas las tablas coinciden entre local y Railway.

Viernes 7 de marzo

Finalización del despliegue y documentación. Migración de los ficheros de imagen al repositorio para su despliegue con Railway. Configuración de EAS Build para la generación de builds nativos (iOS y Android). Intento de generación de IPA sin firma digital para instalación con TrollStore. Creación del script build-ipa.sh para automatizar la compilación en Mac. Redacción del README completo del proyecto con todas las funcionalidades, instrucciones de instalación, despliegue y generación de builds nativos. Redacción de la presente memoria del proyecto.

7. Bibliografía

Acloset. (2024). Acloset - Your Smart Closet. Recuperado de https://acloset.com

Cladwell. (2024). Cladwell - Get Dressed. Recuperado de https://cladwell.com

Combyne. (2024). Combyne - Outfit Ideas. Recuperado de https://combyne.com

Expo. (2024). Expo Documentation. Recuperado de https://docs.expo.dev

Express.js. (2024). Express - Node.js web application framework. Recuperado de https://expressjs.com

Google. (2024). Gemini API Documentation. Recuperado de https://ai.google.dev/docs

Groq. (2024). Groq Cloud API - Whisper. Recuperado de https://groq.com

Jaeger, A. K. (2019). Fashion and the Consumer. Springer International Publishing.

Meta. (2024). React Native Documentation. Recuperado de https://reactnative.dev/docs/getting-started

OpenWeatherMap. (2024). OpenWeatherMap API Documentation. Recuperado de https://openweathermap.org/api

PostgreSQL Global Development Group. (2024). PostgreSQL Documentation. Recuperado de https://www.postgresql.org/docs/

Railway. (2024). Railway Documentation. Recuperado de https://docs.railway.app

Redux. (2024). Redux Toolkit Documentation. Recuperado de https://redux-toolkit.js.org

Smart Closet. (2024). Smart Closet - Your Wardrobe. Recuperado de https://smartcloset.me